



SECURITY AUDIT

5 Pillars

5 ECO

FINAL REPORT

July 2026

BAILSEC.IO



Disclaimer:

Security assessment projects are time-boxed and reliant on information provided by the client, its affiliates, or its partners. The findings in this report should not be considered a comprehensive list of security issues, flaws, or defects in the target system or codebase, and apply only to the specific commit hash, version, or deployment reviewed at the time of the audit. Any subsequent modification, redeployment, or upgrade is outside the scope of this report and may invalidate its findings.

The audit does not constitute investment, legal, financial, regulatory, or tax advice, nor an endorsement of any project or team. It is based on a limited review of materials provided and is delivered on an "as-is," "where-is," and "as-available" basis. Use of blockchain technology is subject to unknown risks and flaws. The scope of this assessment is technical and contract-level; unless explicitly stated otherwise, it does not cover economic or game-theoretic risks, MEV, oracle manipulation, governance attacks, centralization or admin-key risks, off-chain components, front-end code, or the behavior of third-party protocols, dependencies, libraries, or services interacted with by the audited code.

We make no warranties, express or implied, regarding the accuracy, reliability, completeness, or availability of the report or any associated services, and disclaim all implied warranties including merchantability, fitness for a particular purpose, and non-infringement. We assume no responsibility for any product, service, software, code, or material referenced by, linked to, or accessible through the report.

This report is prepared solely for the contract owner and may not be relied upon by any third party. The contract owner is responsible for making their own decisions based on the report and should seek additional professional advice as needed. To the maximum extent permitted by law, our total aggregate liability arising from or related to this audit shall not exceed the fees actually paid for the engagement. The contract owner agrees to indemnify and hold harmless the audit firm and its personnel from any claims, damages, expenses, or liabilities arising from use of or reliance on the report or the audited smart contract.

This disclaimer and any dispute arising from the audit shall be governed by the laws of Wyoming (USA), without regard to its conflict-of-laws principles. By engaging in this audit, the contract owner acknowledges and agrees to these terms.

1. Project Details

Important: Please ensure that the deployed contract matches the source-code of the last commit hash.

Project	Five Pillars – 5 ECO – Audit Report
Website	fivepillarstoken.com
Language	Solidity
Methods	Manual Analysis
Github repository	https://github.com/fivepillarstoken/5ECO/tree/25419087e3609f2e7b59e58af6413b46c26fdbbd
Resolution 1	https://github.com/fivepillarstoken/5ECO/tree/bd3ee33513e8cdef4b84241cf461cc961c007645

2. Detection Overview

Severity	Found	Resolved	Partially Resolved	Acknowledged (no change made)	Failed resolution	Open
High						
Medium	2	2				
Low	8	5	1	2		
Informational	10			10		
Governance						
Total	20	7	1	12		

2.1 Detection Definitions

Severity	Description
High	The problem poses a significant threat to the confidentiality of a considerable number of users' sensitive data. It also has the potential to cause severe damage to the client's reputation or result in substantial financial losses for both the client and the affected users.
Medium	While medium level vulnerabilities may not be easy to exploit, they can still have a major impact on the execution of a smart contract. For instance, they may allow public access to critical functions, which could lead to serious consequences.
Low	Poses a very low-level risk to the project or users. Nevertheless the issue should be fixed immediately
Informational	Effects are small and do not post an immediate danger to the project or users
Governance	Governance privileges which can directly result in a loss of funds or other potential undesired behavior

3. Detection

Disclaimer

Interfaces are not part of the audit and it is expected that these will be fixed before the mainnet deployment. Currently, interfaces do not match with the contract functionalities and as such, compilation will not work.

It is expected that this will be tested and adjusted before deployment.

Airdrop

The **Airdrop** contract is a simple distribution contract which allows the contract owner to distribute arbitrary tokens to different recipients. This can be done via two functions:

- airdrop: single distribution to one recipient
- batchAirdrop: multi-distribution to many recipients

Privileged Functions

- airdrop
- batchAirdrop

Issue_01	Batch distributions revert for single failed transfer
Severity	Informational
Description	The loop in <code>batchAirdrop</code> executes every <code>safeTransferFrom</code> inline with no try/catch and no mechanism to skip or record a failing element. Because all transfers share a single transaction, any one <code>safeTransferFrom</code> that reverts propagates and reverts the entire call, discarding every other transfer in the batch. The incorrect assumption is that each recipient transfer is independent; in reality the batch is strictly all-or-nothing. A revert can arise for reasons outside the owner's immediate control at submission time, most notably a restriction-capable token (USDT blacklist, transfer-restricted or hook tokens) where one recipient is blacklisted or otherwise unable to receive, as well as owner-controllable causes such as a per-element <code>address(0)</code> recipient or zero amount, which are checked inside the loop with <code>revert</code> rather than a skip.
Recommendations	Consider acknowledging this issue.
Comments / Resolution	Acknowledged.

FiveECO

The **FiveECO** contract is a simple ERC20 token which has the following initial mints:

- 17,000,000 5ECO to miningEmissionContract
- 750,000 5ECO to liquidityWallet
- 250,000 5ECO to treasuryWallet
- 1,000,000 5ECO to teamWallet
- 1,000,000 5ECO to rankingWallet
- 1,000,000 5ECO to marketingWallet

It is important to mention that there is no mint function available after contract deployment. The supply is hardcoded to 21 million tokens, deductible by any potential burn via the **ERC20Burnable** inheritance.

Privileged Functions

- none

No issues found.



MiningEmission

The **MiningEmission** contract is inspired by the Synthetix Staking Rewards contract which allows users to burn their SPT tokens via depositing into the InvestmentManager and receiving a reward allocation which is dripped over the duration of 20 years.

Rewards can be claimed via the claimRewards function and after 20 years, the owner can claim any undistributed rewards.

Privileged Functions

- withdrawUndistributedRewards
- setFiveEco

Issue_02	Truncation edge-case can result in DoS of last claimer or governance withdrawal
Severity	Medium
Description	MiningEmission distributes a fixed 17,000,000 5ECO reward budget over 7300 days. Users enter the reward system by calling burnFivePT, which increases their AccountInfo.totalBurned and the global totalBurned. Rewards are tracked through a Synthetix-style rewardPerTokenStored index. Whenever updateReward(account) executes, the contract updates the global reward index, increments totalRewardsAllocated, and optionally checkpoints the selected account's pending rewards. Users later claim their cached rewards through claimRewards. After finishTimestamp, the owner can call withdrawUndistributedRewards, which withdraws the portion of the fixed reward budget that the contract believes was never globally allocated. The key invariant is that the contract's exact TOTAL_REWARD backing must cover both user claims and the owner's final undistributed withdrawal. The root cause is that totalRewardsAllocated and per-user earned use the same rewardPerToken index but floor value at different grouping levels. On every global checkpoint, totalRewardsAllocated increases by: $[(\text{rewardPerTokenStored} - \text{oldRewardPerTokenStored}) * \text{totalBurned}] / \text{PRECISION}$ This floors the allocation per global reward-index update. However, a user's reward is calculated as: $\text{account.totalBurned} * [\text{rewardPerToken}() - \text{account.rewardPerTokenPaid}] / \text{PRECISION}$

A user can remain uncheckpointed across multiple global index updates. When that user later claims, their reward is floored once over the aggregated rewardPerToken difference rather than being floored separately for every global interval.

This creates a normal integer division mismatch:

$\text{floor}(x) + \text{floor}(y)$ can be less than $\text{floor}(x + y)$

As a result, cumulative user claims can exceed totalRewardsAllocated by dust.

Impact 1:

A late user claim can permanently revert by dust if the owner has already withdrawn the full computed TOTAL_REWARD - totalRewardsAllocated.

Impact 2:

The owner's withdrawUndistributedRewards() can permanently revert by dust if users claim first and cumulative claims exceed totalRewardsAllocated.

Impact 3:

totalRewardsAllocated is not an exact liability cap. Off-chain accounting that assumes $\text{totalRewardsAllocated} + \text{undistributedRewards} == \text{TOTAL_REWARD}$ fully covers user claims can be wrong by rounding dust.

TOTAL_REWARD = 17,000,000 SECO

DURATION = 630,720,000 seconds

PRECISION = 1e18

#

t = 831:

User A burns 1.000295256936867810 fivePT

t = 404,400:

User B burns 1.000537635120628154 fivePT

t = 5,448,663:

User A claims once

t = finishTimestamp:

Final reward state is updated and both users' full lifetime claims are considered

#

Both burn amounts are above 1e18, and the second burn is more than the InvestmentManager 4-hour depositDelay after the first one, so this example does not rely on the queued-deposit issue.

At the end:

totalRewardsAllocated

= 16,999,977.601788432267884318 5ECO

Total user claims

= 16,999,977.601788432267884319 5ECO

Owner claim

TOTAL_REWARD - totalRewardsAllocated

= 22.398211567732115682 5ECO

#

Total user claims + owner-computed undistributed withdrawal

	= 17,000,000.00000000000000000001 5ECO
Recommendations	Consider donating some dust 5ECO to the contract upon deployment.
Comments / Resolution	Resolved, the client will manually seed the contract with dust.

Issue_03	InvestmentManager interface omits <code>poolRewardPerInvestorPaid</code> struct field
Severity	Informational
Description	The <code>InvestmentManager.InvestorInfo</code> struct used by <code>MiningEmission</code> is not a complete copy of the real <code>InvestmentManager.InvestorInfo</code> struct. It omits the <code>uint256[9] poolRewardPerInvestorPaid</code> field which is used internally by <code>InvestmentManager</code> for pool reward accounting. If pool reward data is needed later, the interface must expose the omitted <code>poolRewardPerInvestorPaid</code> field.
Recommendations	Consider acknowledging this issue.
Comments / Resolution	Acknowledged.

Issue_04	Decoupling of reward eligibility and start produces inherent owner claims
Severity	Low
Description	The root cause is a decoupling between the emission clock and reward eligibility. <code>finishTimestamp</code> (and the accrual baseline <code>updatedAt</code>) are fixed at deployment, but the ability to earn is gated behind <code>setFiveECO</code>: <code>burnFivePT</code> reverts with <code>FiveECONotSet</code> while <code>fiveECO == address[0]</code> During the forced window <code>[startTimestamp, setFiveECO]</code>, and, more generally, the entire interval <code>[startTimestamp, firstBurnTime]</code>, no user can hold a position. The reward engine compounds this: <code>rewardPerToken()</code> short-circuits and returns <code>rewardPerTokenStored</code> unchanged whenever <code>totalBurned == 0</code>, while <code>updateReward</code> still advances <code>updatedAt</code> to the current clamped timestamp. As a result, when the first burn eventually occurs, <code>updatedAt</code> has already been moved forward to that moment (before <code>totalBurned</code> is incremented), so the streaming budget for the entire pre-first-burn interval is neither credited to the first burner nor added to <code>totalRewardsAllocated</code>. The linear stream nonetheless keeps advancing toward the deployment-anchored <code>finishTimestamp</code>, so that skipped budget is permanently excluded from participant distribution and remains inside <code>TOTAL_REWARD - totalRewardsAllocated</code>. There is no compensating logic that re-anchors <code>finishTimestamp</code> to the first burn or that folds the skipped interval into the first participant's accrual.
Recommendations	Consider setting the 5ECO token during deployment or accepting this window.
Comments / Resolution	Acknowledged.

Issue_05	MiningEmission does not queue all temporary InvestmentManager deposit failures
Severity	Low
Description	MiningEmission only queues deposits when the InvestmentManager deposit delay is still active. If that check passes, it immediately calls investmentManager.deposit() <pre> //InvestmentManager.InvestorInfo memory investorInfo = investmentManager.accountToInvestorInfo(address(this)); if (investorInfo.lastDepositTimestamp + investmentManager.depositDelay[] > block.timestamp) { amountWaitingToBurn += amount; emit AmountWaitingToBurnIncreased(amount); } else { uint256 totalAmount = amount + amountWaitingToBurn; amountWaitingToBurn = 0; fivePT.approve(address(investmentManager), totalAmount); investmentManager.deposit(totalAmount, address[0]); } </pre> However, InvestmentManager.deposit() can revert due to the InvestmentManager may not have reached startTimestamp yet, or a pool criteria update may be active → <pre> modifier NotInPoolCriteriaUpdate { if (isUpdateCriteriaActive) revert PoolCriteriaUpdateNotEnded(); _; } function deposit(uint256 amount, address referer) external NotInPoolCriteriaUpdate { address investorAddress = _msgSender(); InvestorInfo storage investor = accountToInvestorInfo[investorAddress]; </pre>

	<pre> if { block.timestamp < startTimestamp block.timestamp - investor.lastDepositTimestamp < depositDelay } revert DepositNotYetAvailable[]; </pre> The transaction reverts instead of being queued, even though MiningEmission already has a queue mechanism for delayed deposits.
Recommendations	Handle all expected temporary InvestmentManager unavailability states, not only depositDelay. Usually it is expected that governance ensures this, since the client is also responsible for the InvestmentManager contract.
Comments / Resolution	Resolved.

Issue_06	InvestmentManager rewards earned by MiningEmission cannot be claimed or distributed
Severity	Informational
Description	MiningEmission deposits queued or immediate fivePT into InvestmentManager: <pre> if (investorInfo.lastDepositTimestamp + investmentManager.depositDelay[] > block.timestamp) { amountWaitingToBurn += amount; emit AmountWaitingToBurnIncreased(amount); } else { uint256 totalAmount = amount + amountWaitingToBurn; amountWaitingToBurn = 0; fivePT.approve(address(investmentManager), totalAmount); investmentManager.deposit(totalAmount, address[0]); if (totalAmount > amount) { emit WaitingToBurnFlushed(totalAmount - amount); } } </pre> In InvestmentManager, the investor is always msg.sender, so those deposits accrue daily, referral, and pool accounting to address(MiningEmission). InvestmentManager has a claimReward() function, but MiningEmission has no function that calls it. Since InvestmentManager.claimReward() claims only for msg.sender, no EOA or owner can claim rewards on behalf of the MiningEmission contract and those rewards earned by the MiningEmission are lost forever.
Recommendations	Implement the claimReward() functionality for the InvestmentManager.
Comments / Resolution	Acknowledged.

Issue_07	MiningEmission does not account for deposit fees
Severity	Informational
Description	MiningEmission credits users with the full amount passed to burnFivePT(): <pre>function burnFivePT(uint256 amount) external nonReentrant updateReward(msg.sender) { if (address(fiveECO) == address(0)) revert FiveECONotSet(); if (amount == 0) revert InvalidAmount(); if (block.timestamp >= finishTimestamp) revert MiningEmissionFinished(); fivePT.safeTransferFrom(msg.sender, address(this), amount); accountInfo[msg.sender].totalBurned += amount; totalBurned += amount;</pre> The actual InvestmentManager.deposit() applies a deposit fee and only burns/credits the net toInvestor amount. This means MiningEmission reward shares are based on total fivePT tokens taken from the user, while InvestmentManager's actual totalDeposit and burn are based on net tokens after the deposit fee.
Recommendations	Consider acknowledging the issue.
Comments / Resolution	Acknowledged.

Issue_08	Violation of CEI
Severity	Informational
Description	One or more parts in the code are violating the checks-effects-interactions pattern by executing an external call before checks and effects are applied.
Recommendations	Consider following the CEI pattern.
Comments / Resolution	Acknowledged.

FounderPackages

The **FounderPackages** contract is the protocol's founder package sale, referral, and vesting contract. It allows users to buy predefined **fiveECO** founder allocations by paying **USDT**, while the purchased **fiveECO** is not transferred immediately. Instead, each purchase creates an individual **WithdrawInfo** vesting schedule for the buyer.

The contract is phase-based. Each sale phase defines its own price, start time, duration, lock multipliers, and package list. The owner must pre-fund **fiveECO** collateral for each phase when creating or editing a phase.

Appendix: Staggered Amount

`staggeredAmount()` implements a weekly stepped vesting schedule after the configured lock period has elapsed. No amount is releasable before start, and at exactly start the releasable amount is still zero because no full week has passed. During the vesting period, the vested amount increases only in whole-week increments, and at start + duration the full remaining allocation becomes vested.

Appendix: Referral Application

FounderPackages applies referral rewards as a split of the buyer's gross USDT payment, not as an additional fee on top of the purchase price. When a buyer has a registered referrer, the contract pays up to four uplines according to [800, 400, 200, 100] bps and subtracts the paid referral amount from the amount sent to `usdtReceiver`. If the referral chain has fewer than four uplines, the unpaid referral levels are not stored or reserved; they remain part of the receiver's proceeds.

Privileged Functions

- `setUSDTReceiver`
- `setWithdrawDuration`
- `createPhase`
- `editPhase`
- `withdrawFromPhase`

Issue_09	Purchase has no phase or payment slippage protection
Severity	Medium
Description	<code>purchase()</code> does not let the buyer specify the expected <code>phaseId</code>, maximum acceptable USDT payment, minimum expected <code>fiveEco</code> allocation, or a deadline. At execution time, the contract calls <code>_getActivePhaseId()</code> and purchases from whichever phase is active in that block. If a user's transaction is delayed across a phase boundary due to any reason then <code>purchase()</code> can execute against a different phase than the user intended and the buyer might pay more USDT than expected.
Recommendations	Consider adding slippage checks else consider acknowledging the issue.
Comments / Resolution	Resolved.

Issue_10	<code>withdrawDuration</code> change can influence already broadcasted purchases
Severity	Low
Description	At line 404, purchase reads the mutable global <code>withdrawDuration</code> at the moment of execution and passes it straight into <code>_createWithdrawSchedule</code>, which stores it as the buyer's <code>WithdrawInfo.duration</code>. The buyer has no way to express or bound the vesting duration they are willing to accept: purchase takes only <code>packageType</code>, <code>lockPeriod</code>, and <code>referrer</code>, there is no minimum/maximum-duration argument and no deadline, and the value is never snapshotted at buyer-intent time. Consequently the terms a buyer sees when they sign a purchase are not the terms guaranteed to them: an owner <code>setWithdrawDuration</code> transaction that is included ahead of a pending purchase (whether an intentional frontrun using higher gas after observing the mempool, or an unintended race where the config is updated between the buyer's decision and inclusion) causes that buyer's schedule to be created under the new duration.
Recommendations	Consider communicating a change of the <code>withdrawDuration</code> upfront
Comments / Resolution	Resolved.

Issue_11	Unbounded package iteration can permanently block unsold collateral withdrawal
Severity	Low
Description	<code>withdrawFromPhase()</code> processes every package in a phase in a single transaction. For each package, it reads the package cap, reads sold amount, writes the package as fully sold, emits an event, and only then transfers the unsold <code>fiveEco</code> amount to the owner. If a phase contains enough package tiers, this function can exceed the block gas limit and always revert. Because the function is all-or-nothing the unsold collateral for that phase can become permanently inaccessible.
Recommendations	Consider processing a bounded package range per transaction else consider acknowledging the issue.
Comments / Resolution	Partially resolved, it has to be mentioned that the full package array is casted into memory. The loop has been adjusted.

Issue_12	Lock factors are not required to increase with longer lock duration
Severity	Low
Description	<code>LOCK_PERIODS</code> are ordered as 1, 2, and 3 years, and <code>purchase()</code> uses the selected <code>lockFactor</code> to compute the buyer's <code>fiveEcoAmount</code>. <code>_checkPhaseInfo()</code> only validates that each factor is at least <code>PCT_BASE</code>; it does not require <code>lockFactors[1] > lockFactors[0]</code> or <code>lockFactors[2] > lockFactors[1]</code>, this should be enforced because higher locked periods should have a higher lock factor.
Recommendations	Make sure lock factors increase with lock periods.
Comments / Resolution	Resolved.

Issue_13	Referrals can always be routed to self-owned address
Severity	Low
Description	The anti-self-referral protection is under-specified. <code>_registerReferrer</code> rejects only the case where the referrer is the buyer's own address (<code>if (referrer == account) revert InvalidReferrer();</code>). It does not require the referrer to be an independent, pre-existing, or economically-committed participant, and no other check constrains who the referrer may be. A buyer who controls a second address B simply passes B as referrer on their first purchase. <code>_registerReferrer</code> accepts it (<code>B != A</code>), and <code>_checkRefererCirculation[B]</code> passes because B has no upline (<code>_accountToReferrer[B] == address(0)</code>), so the loop breaks on the first hop without detecting circulation. From then on, every purchase from A triggers <code>_distributeReferralFee</code>, which pays the level-1 fee (8% of the package price) to B and reduces the amount forwarded to <code>usdtReceiver</code> by that same fee. Because B is controlled by the buyer, the fee is not a real cost to them, it is recovered in full, so the net effect is a permanent 8% discount on every purchase, financed by protocol revenue rather than by a genuine referrer.
Recommendations	Consider acknowledging this issue. Sybil attacks for referral systems are generally hardly fixable.
Comments / Resolution	Acknowledged.

Issue_14	Many scheduled future phases can make current phase purchases run out of gas
Severity	Low
Description	<code>purchase()</code> calls <code>_getActivePhaseId()</code>, which starts at <code>nextPhaseId</code> and walks backward until it finds a phase active at the current block timestamp. This means purchase gas grows with the number of already created future phases after the currently active phase. If enough future phases are scheduled, every purchase during the current phase can exceed the block gas limit and revert.
Recommendations	Consider maintaining a current phase pointer.
Comments / Resolution	Resolved. It has to be noted that the <code>startPhaseId</code> parameter is merely a marker for the expected phase to determine the backwards loop and not related to the actual phase.

Issue_15	DoS of purchase due to blacklisted referrer
Severity	Low
Description	<code>FounderPackages</code> sells vested <code>fiveECO</code> in sequential phases; buyers pay USDT through <code>purchase(packageType, lockPeriod, refferer)</code>. On a buyer's first purchase the contract registers a referrer via <code>_registerReferrer</code>, writing <code>_accountToReferrer[msg.sender]</code> exactly once and treating it as immutable thereafter. On every purchase where the buyer has a non-zero referrer, the contract deducts referral fees from the buyer's USDT and pays them out along the upline in <code>_distributeReferralFee</code>. That helper walks up to <code>REF_LEVEL_COUNT = 4</code> referrers, computing each level's fee from <code>REF_SYSTEM_FEES = [800,400,200,100]</code> and pushing it with <code>usdt.safeTransferFrom(msg.sender, referrer, refFee)</code> directly to each upline address before the net amount is forwarded to <code>usdtReceiver</code> and the buyer's vesting schedule is created. The referral payout is therefore an inline, mandatory dependency of the entire purchase flow: purchase liveness is coupled to the ability of every upline address to receive USDT.
Recommendations	Consider acknowledging this issue.
Comments / Resolution	Resolved. It has to be noted that <code>safeTransferFrom</code> was replaced with <code>transferFrom</code> which is called directly and expects a bool return. This may break on other chains or token implementations where USDT reverts on failure and does not return a bool, or with weird token implementation. This should be adjusted. For example, USDT on ETH mainnet does not return a boolean, which means transfers will revert. This is however not an issue because this contract is meant to be deployed on BSC.

Issue_16	Violation of CEI
Severity	Informational
Description	One or more parts in the code are violating the checks-effects-interactions pattern by executing an external call before checks and effects are applied.
Recommendations	Consider following the CEI pattern.
Comments / Resolution	Acknowledged.

Issue_17	Package pricing assumes compatible token decimals and can misprice USDT payments
Severity	Informational
Description	The purchase formula computes the payment amount as: $usdtAmount = phase.price * package.quantity / 10 ** PRICE_DECIMALS;$ <code>package.quantity</code> is also used directly in the <code>fiveEcoAmount</code> calculation and the fiveEco token has 18 decimals. The formula does not normalize by <code>fiveEco.decimals()</code> or <code>usdt.decimals()</code>. It only works correctly if <code>package.quantity</code> and the USDT payment token share the same decimals expected by <code>phase.price</code>. If the configured USDT token uses 6 decimals then 5ECO quantities become mispriced.
Recommendations	If the contracts would be deployed on chains other than BSC then this should be fixed and decimals should be normalised, else consider acknowledging the issue.
Comments / Resolution	Acknowledged.

Issue_18	Inconsistency in lock duration documentation and real appearance
Severity	Informational
Description	The lock durations are hardcoded as <code>LOCK_PERIODS = [360 days, 720 days, 1080 days]</code> and documented as “Lock period options in seconds [1, 2, and 3 years].” A calendar year is 365 days (365.25 accounting for leap years), so 360/720/1080 days encode a 360-day “year,” not a calendar year. The incorrect assumption is the equivalence “1 year == 360 days” baked into the constant while the human-facing documentation promises full years. Because the value flows unmodified into <code>_createWithdrawSchedule</code> as <code>start = block.timestamp + LOCK_PERIODS[lockPeriod]</code>, every schedule’s release begins earlier than a strict calendar-year interpretation implies: 5 days early for the 1-year tier, 10 days for the 2-year tier, and 15 days for the 3-year tier (slightly more once leap days are counted).
Recommendations	Consider acknowledging this note.
Comments / Resolution	Acknowledged.

Issue_19	Referral fee might round to zero
Severity	Informational
Description	in <code>_distributeReferralFee</code>, <code>refFee = amount * REF_SYSTEM_FEES[i] / PCT_BASE</code> truncates; for <code>usdtAmount</code> small enough (e.g. <code>usdtAmount <= 12</code> makes the 800-bp level-1 fee floor to 0) every level yields <code>refFee == 0</code>, the <code>if (refFee > 0)</code> guard skips the transfer/event, yet the referrer relationship is still traversed and the buyer still receives full <code>fiveEcoAmount</code>. Real and reachable but bounded to dust-priced package configurations, hence non-material.
Recommendations	Consider acknowledging this note.
Comments / Resolution	Acknowledged.

Issue_20	Missing validation in poolCap can result in unsellable packages
Severity	Informational
Description	FounderPackages sells founder packages in sequential, owner-configured phases. For each phase the owner supplies a list of PackageInfo tiers via createPhase (or editPhase), each tier defining a base quantity of fiveECO, a bonus percentage, and a poolCap that bounds the gross fiveECO that may be sold for that tier. The owner funds the phase by transferring collateral equal to the sum of all poolCap values. Buyers then call the permissionless purchase, selecting a packageType and a lockPeriod; the contract computes the buyer's fiveEcoAmount from the tier quantity, the phase lockFactors for the chosen lock period, and the tier bonus, then records it against the tier's running sold total. The per-tier sold total is capped by poolCap through the PackagePoolCapExceeded gate, and any unsold collateral is returned to the owner after the phase ends via withdrawFromPhase. The poolCap field is thus the accounting surface that both bounds sales and sizes the required collateral, so its configuration directly determines whether a tier can be sold at all. In cases where the poolCap is below the quantity (including the bonuses), all sales would revert. Similarly, a problem will arise if the aggregation of the quantities is not aligned with the poolCap.
Recommendations	Consider making sure that the poolCap is always a reasonable value.
Comments / Resolution	Acknowledged.